VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Computer Graphics

Progress Report

Assignement 1

Advisor(s):	Trần Thị Ngọc Trâm	
Student(s)	Lê Công Minh Khang	2252295
	Philippe Belda	2660010

HO CHI MINH CITY, 26 OCTOBER 2026

Table of Contents

I. Introduction 3
II. Software structure 3
II.1. Objects 4
II.2. Scenes 5
III. Shape Generators 6
III.1. Data format 7
III.2. Obj files 8
III.3. 2D shapes 9
III.3.1. `ngonGenerator` helper function 9
III.3.2. Simple 2D shapes (triangle, rectangle, trapezoid, arrow) 9
III.3.3. n-gon shapes (pentagon, hexagon, circle, ellipse) 9
III.4. 3D shapes 10
III.4.1. `quads2triangles` helper function 10
III.4.2. `normalsGenerator` helper function 10
III.4.3. `uvSlicer` helper functions 11
III.4.4. Cube 12
III.4.5. Simple `ngonGenerator` and `quads2triangles` shapes (cylinder, cone, truncated cone, tetrahedron, prism) 13
III.4.6. Sphere 14
III.4.7. Torus 14
III.4.8. Surface 15
IV. Shaders 16
V. Lighting 16
VI. GUI 17
VII. Challenges encountered 18

I. Introduction

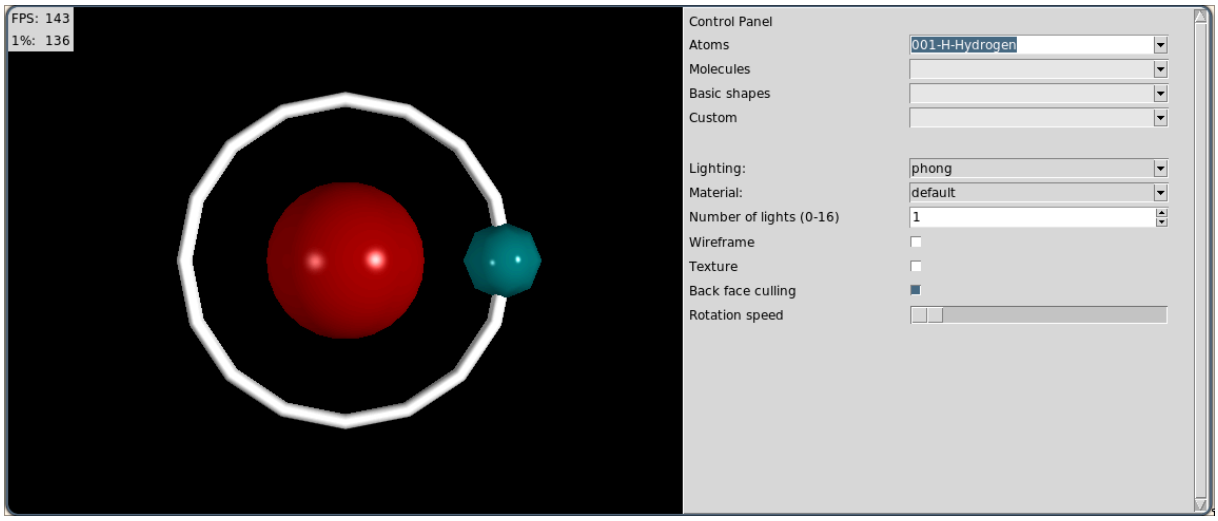


Figure 1: Application displaying an hydrogen atom

This project aims to provide an interface for displaying 3D scenes. A scene is defined as a collection of objects and lights. The user can create its own scenes through code via the interface provided (see Section II.2). Some are available by default, allowing the user to visualize atoms following the Bohr model, molecules and some basic shapes in 2D and 3D. The user has the ability to modify the aspect of the objects by modifying the lighting program used, and the materials of objects in the scenes.

II. Software structure

The software has been structured to ensure modularity and concern separation. The GUI module is tasked to set the scene informations (objects to be rendered, different parameter, …). This module mainly defines the GUI elements with Tkinter, creating GL context and object caching in order to reuse similar objects between scenes. Another module, named pipeline, is responsible for uploading data to GPU and drawing the scene selected.

In Figure 2 is provided a more detailed diagram to better understand the sequence of operations and the responsibilities of each module.

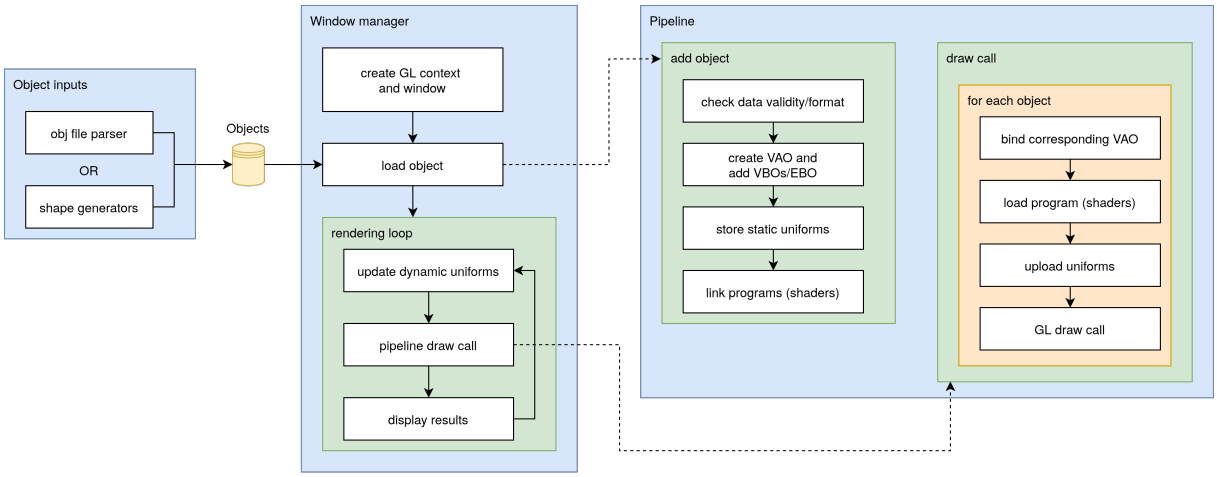


Figure 2: Diagram of the software structure

Our application initializes with an empty scene, with no objects displayed. The user can then select one of the registered scenes. To allow better performances and a fluid user experience, objects from the different scenes are cached by the window manager module. This results in short waiting time when changing scenes. The process is detailed in Figure 3.

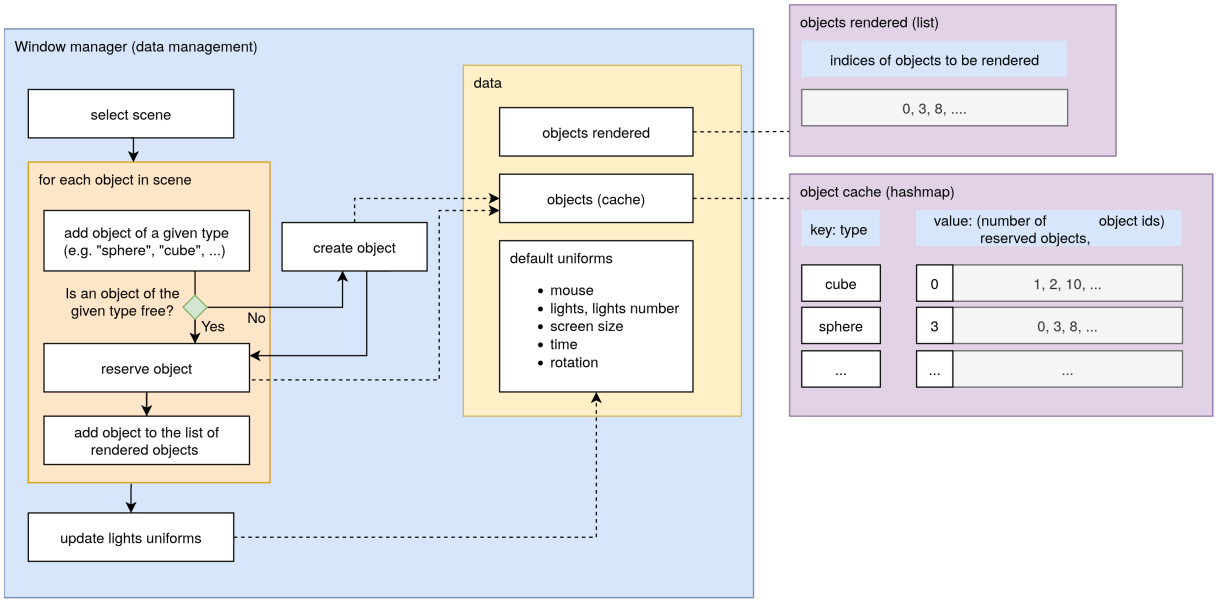


Figure 3: Details of the scene selection

II.1. Objects

In this project, objects are defined by their attributes:

- mesh: a mesh composed of vertices, normals and colors, stored in a VAO (Vertex Array Object)
- program: the program used to render the object. A single program can be used to render several objects.

- uniforms: as the programs are shared between objects, an objects can redefine some uniforms of the program to adapt the rendering

II.2. Scenes

Scenes are a collection of objects. The objects positions in the world are changed thanks to the `model` matrix uniform.

All scenes are defined under the `./scenes/` directory. The applications proceeds to scan it at startup to discover the scenes. The user can define its own scenes inside this directory, but sould not modify the `./scenes/default/` directory where are stored the default scenes (basic shapes, atoms and molecules).

To define a new scene, the user must declare a function named `scenes` that takes a `AppOgl` argument, and returns a dictionary of couples <scene name, `Scene` instance>. The `Scene` instances have their function `build_scene` overloaded, and this is where the definition of the scene takes place, with the functions provided by the `Scene` class.

Function name	Function effect

`add_object(`
`  self,`
`  name: str,`
`  filePath: str = None,`
`  data_f: lambda: dict = None,`
`)`
	Add an object of given name. If an object of this name is in memory and available, it is reused.
	• `name` is the name of the object
	• If `filePath` is provided, the file is parsed (must be an `.obj` file) and added to the scene. No caching is done on files.
	• If `data_f` is provided, the data used to create the object will be obtained from this function. Warning: if an object is cached and available with the same name, this parameter will be ignored

`update_object(`
`  self,`
`  obj_id,`
`  program_name=None,`
`  model_matrix=None,`
`  mode=None,`
`  static_uniforms=None,`
`  textures=None,`
`  color_mode=None,`
`  material=None,`
`)`
	Updates the data of the object of index `obj_id`. The user should use this function after `add_object` to ensure the object has the wanted properties: a cached object can have been returned, thus having modified properties from the initial object.

`create_program(self,`
`  name,`
`  vert_shader=None,`
`  frag_shader=None,`
`)`
	Creates a shader program named `name`. If `name` is a predefined shader program, no need to fill `vert_shader` and `frag_shader` paths.
	Note: The predefined shader programs (flat, interpolation, phong and goureaud) will be automatically created, no need to call this function for

	them. Returns the id of this program.

`add_lights(`
`self,`
`nb_lights,`
`uniforms=None,`
`)`
	Add lights to the scene. Takes a list of `Light` class instances.

The user can find examples on how to define scenes in the `./scenes/` directory.

III. Shape Generators

All shape generators follows a similar process like the following graph, with commonly used algorthms and tools having slightly different applications.

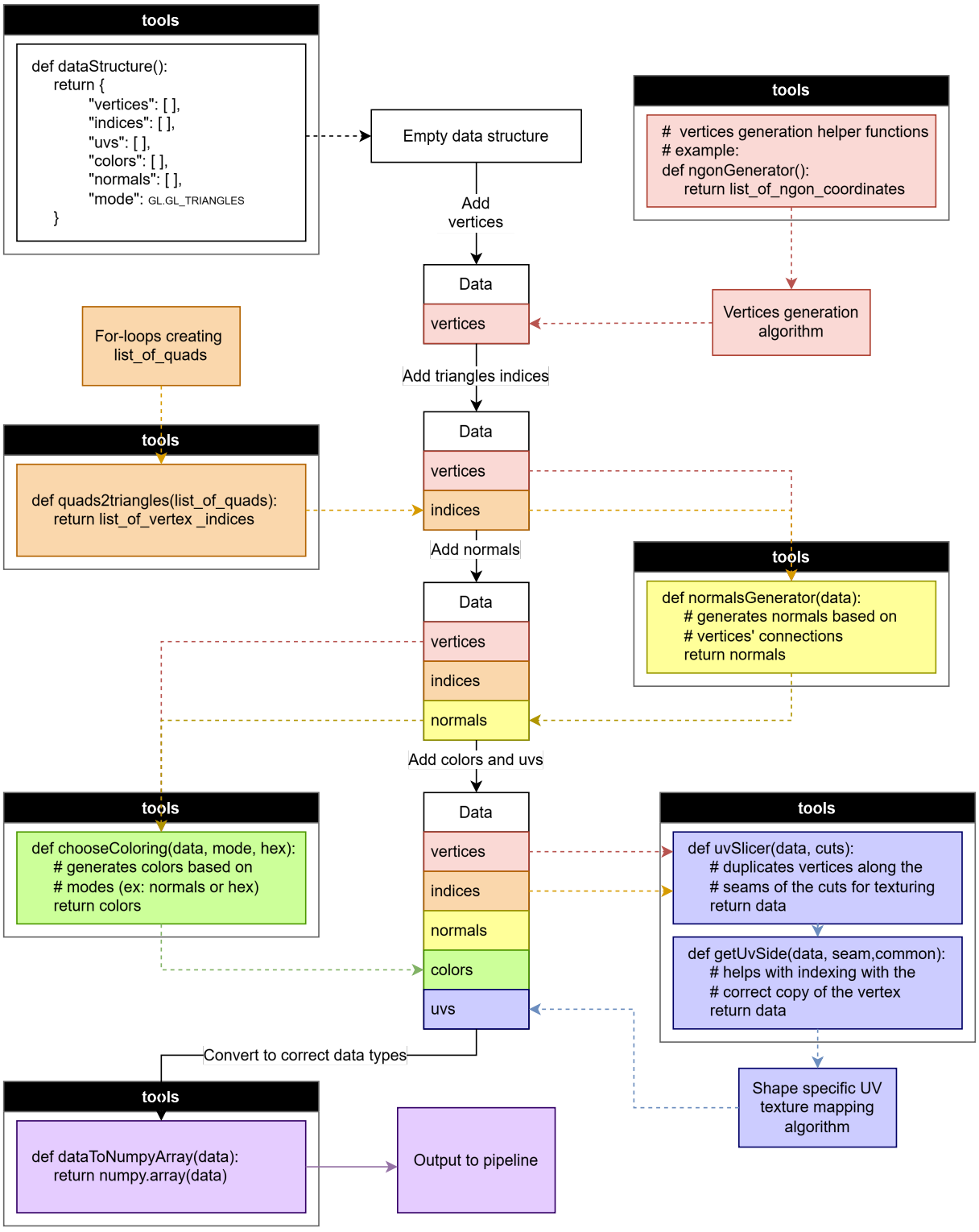


Figure 4: General shape generation flow

III.1. Data format

We chose to bundle object data for rendering under the form:

```
data = {
  "vertices" # np.array((n, 3))
  "indices"  # (optional): np.array((n)) if ommited, will be picking k-tuples
from the vertices array where k is the size of primitive (eg 3 for triangles)
```

```
  "colors"   # np.array((n, 3)) each component must be in [0, 1]
  "uvs"      # np.array((n,2)) each component must be in [0,1]
  "normals": # np.array((n, 3))
  "mode":    # GL mode, indicating the rendering mode (eg: GL\_TRIANGLES,
GL\_TRIANGL_STRIP, ...)
}
```

All shape generators and `.obj`/`.mtl` files parsing outputs are converted to this format before being passed into the rendering pipeline:

- `vertices` and `indices` are generated/parsed per shape/file.
- `colors`:
  - from `.mtl` file parsing, for simplicity we only parse the materials' `Kd` field as flat vertex colors
  - from generators, we have a few modes useful for debugging/making scenes:
    - `hex`: pass in a hex value for RGB colors (ex. `0xff0000` will color the model red)
    - `normal`: taking in each vertex's 3 `normals` values as red green blue values, useful to aid in understanding the shape's surface
    - `position`: taking in each vertex's coordinate scaled such that the model size goes from [0,1] in the three axis as color.
    - `modulo`: taking in each vertex's coordinate arbritrarily multiplied, modulous-ed, and normalized to fit on [0,1] in the three axis as color. Early implementation for testing.
- `uvs`: asside from file parsing in and used as is, it is complicatedly generated and has a process we will go into later.
- `normals`: generated from `vertices` and `indices` using an algorithm

III.2. Obj files

From the wiki, we can see that `.obj` files are text files with the syntax:

```
<fieldtype> <field1> <field2> <etc..>
v 0.123 0.234 0.345 1.0
...
vt 0.500 1 [0]
...
vn 0.707 0.000 0.707
...
vp 0.310000 3.210000 2.100000
...
f 1 2 3
f 3/1 4/2 5/3
f 6/4/1 3/5/3 7/6/5
f 7//1 8//2 9//3
f ...
```

Things to note of is that this files contains `vertices`, `uvs`, `normals` with fields starting with `v`, `vt`, `vn`. Also `f` fields denotes `indices`, in the format of having multiple `v/vt/vn` field per vertex index that is in a face, with the indices being 1-indexed. Colors are parsed from the `.mtl` files that denotes what material will be applied the next group of faces have. The parsing goes as follows:

- We first parse `v`, `vt`, `vn` into a dictionary `fromFile`.
- For every index fields we can parse in an `f`, we check if that tripplet exists already in a dictionary `tripplets`:
  - if it already exists, we append the corresponding index to the output `indices`
  - otherwise, we append to the `vertices`, `uvs`, `normals`, and the currently set `colors` and take the newly appended position to store in `tripplets`, we append the index to the output `indices`

The result is a properly formated `data` object to be passed into the pipeline.

III.3. 2D shapes

III.3.1. `ngonGenerator` helper function

Generating circles comes up a few times in some of the 2D and 3D shapes generation methods, thus a helper function like `ngonGenerator` is very convenient.

The exact functionality and implementation varies depending on use cases, which is why the function is copied accross files instead of declared and imported. In general, it takes in:

- `n`: number of vertices generated
- `size`: the distance the vertices will be from (0, 0, z)
- `z`: the z coordinate of vertices

It follows:

Given step size in angle:

$$\theta = \frac{2\pi}{n}$$

for i in range(n) we create a new vertex at coordinate:

$$(\cos(i\theta) \times \text{size}, \sin(i\theta) \times \text{size}, z)$$

III.3.2. Simple 2D shapes (triangle, rectangle, trapezoid, arrow)

These shapes are relatively simple, so each generator only takes as parameters a few basic arguments to scale them in different aspects. The vertices then gets their coordinate multiplied accordingly and populating `data["indices"]` is hardcoded.

III.3.3. n-gon shapes (pentagon, hexagon, circle, ellipse)

All of these shapes can be generated with some variation of the `ngonGenerator` function. Notably:

- Pentagon: n = 5
- Hexagon: n = 6
- Circle: n = 60
- Ellipse: n = 60, with a multiplier to the width of the n-gon in one of the axis

How these rings of vertices used to get triangulated is that they all have 1 common vertex, and then iterate around the n-gon for the other 2 vertices. This method is simple and ensures the correct ordering of the indices but can leads to rendering artifacts around the common vertex.

How they are currently triangulated is adding an extra vertex in the center of the `n-gon` and manually implementing the GL_TRIANGLE_FAN mode with GL_TRIANGLES.

III.4. 3D shapes

III.4.1. `quads2triangles` helper function

A lot of 3d shapes are more conveniently generated as quads (4 sided polygon). However, to be able to render in mode GL_TRIANGLES, we need to convert these lists of quads into a list of triangles.

It follows:

```
#      0.----.3
#       |\   |
#       | \  |
#       |  \ |
#       |   \|
#      1'----'2
# inputQuad = [0,1,2,3]
res = []
for quad in listOfQuads:
  res.append([[quad[0],quad[1],quad[2]]])
  res.append([[quad[0],quad[2],quad[3]]])
return res
```

Note that the method requires that quads part of the input list needs to have vertices indexed in a counter clockwise direction. Usage of this method would need some consideration in how the listOfQuads are generated.

III.4.2. `normalsGenerator` helper function

Normals generation follows the graph below.

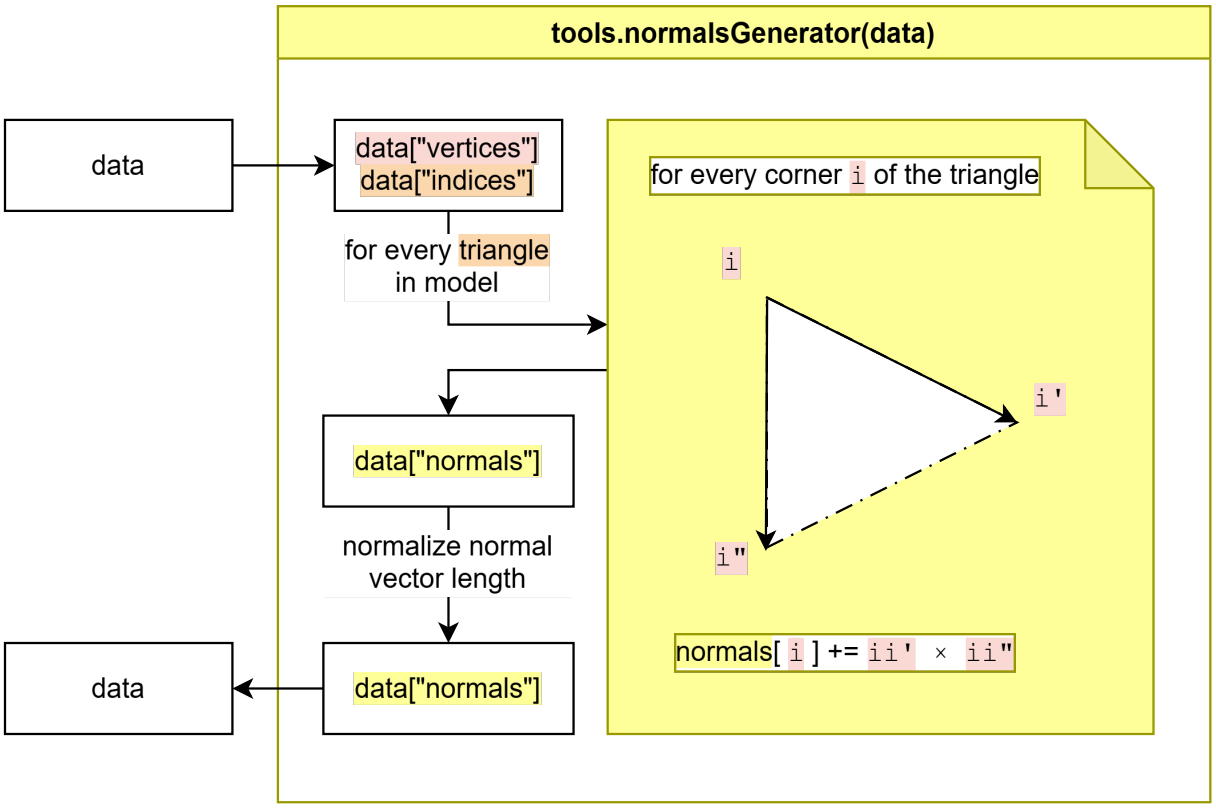


Figure 5: Normals generator

One notible consequence of this algorithm is that vertices that are part of a lot of faces will have its normal heavily biased towards whatever the largest group of faces that shares normal direction. This would encourage the model designers have its `vertices` be evenly spread and prefer to have `indices` mostly tile the surface than concentrate around 1 vertex

III.4.3. `uvSlicer` helper functions

A simple over view of the slicer algorithm is as follows

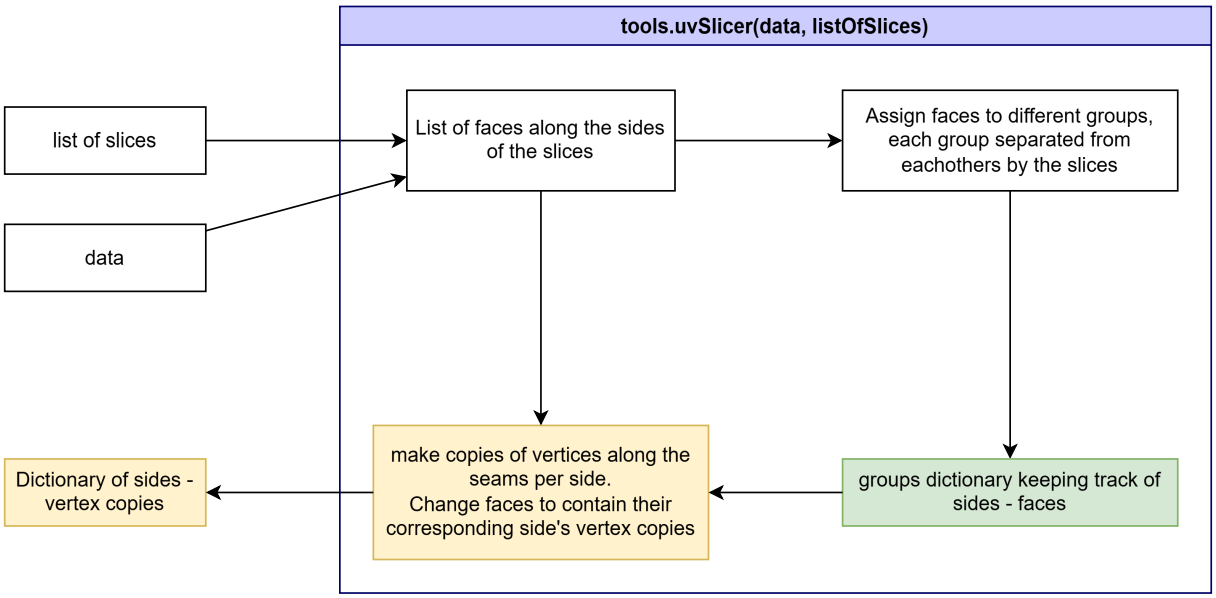


Figure 6: Slicer diagram (simple)

The gist is that with the output `isTheSame` the model designer can thoughtfully plan slices on the model surface to beable to map the same vertex at multiple uv coordinates.

A more in-depth look into how the algorithm works is not important to the system as a whole, but a detailed diagram is provided as follows:

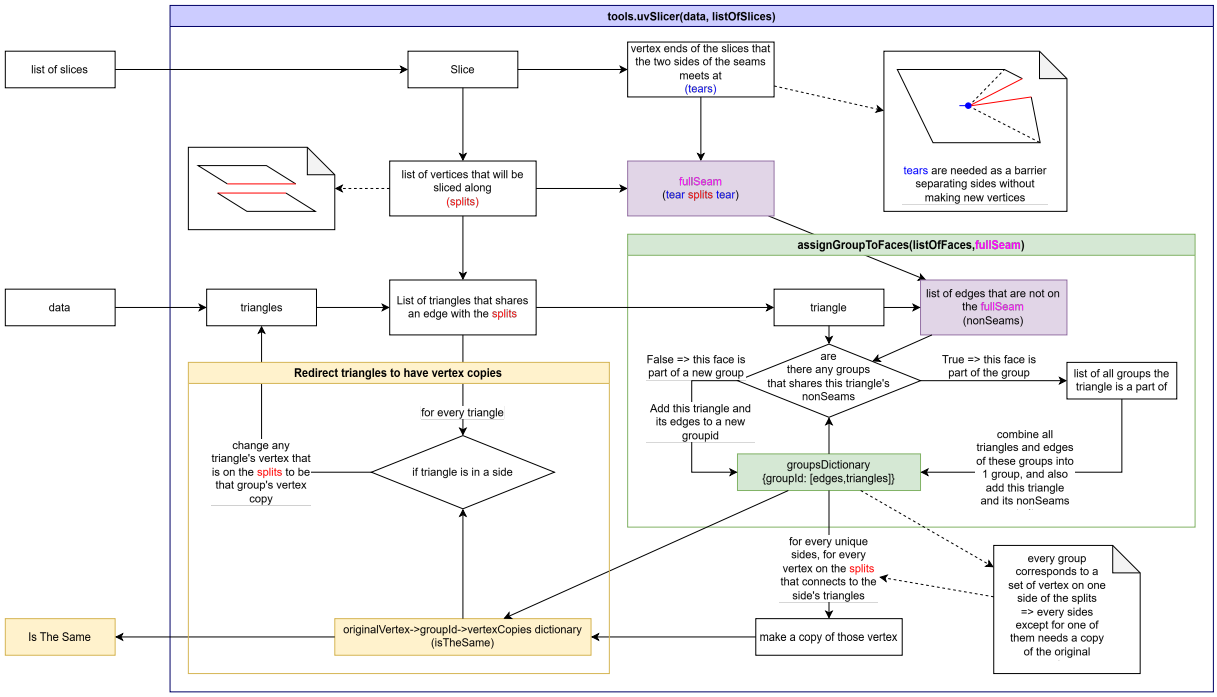


Figure 7: Slicer diagram (detailed)

III.4.4. Cube

The simplest 3d shape, establishes the generation pattern that is used futher down the line.

```
p = size/2
obj["v"].append([ -p ,  p ,  p])
obj["v"].append([ -p ,  p , -p])
obj["v"].append([ -p , -p ,  p])
obj["v"].append([ -p , -p , -p])
obj["v"].append([  p , -p ,  p])
obj["v"].append([  p , -p , -p])
obj["v"].append([  p ,  p ,  p])
obj["v"].append([  p ,  p , -p])

listOfQuads = []
# top
listOfQuads.append([0,2,4,6])
# bottom
listOfQuads.append([7,5,3,1])

for i in range(0,8,2):
  listOfQuads.append([(i+0)%8,(i+1)%8,(i+3)%8,(i+2)%8])
```

We initialize `vertices` in pairs that creates 4 parallel edges of the cube going counter clockwise (in the code shown above it is parallel to the z axis).

Then quads of the top and bottom faces of the cube are hard coded, paying attention to "reverse" the order for the bottom face to ensure that it faces the correct way (backface culling).

Finally, as the order of vertex declaration is suitable, we can use a loop to iterate over the 4 side faces of the cube, with modulo operation `%` to be able to connect the last vertex pair to the first.

For `uvs`, the slicing is chosen to beable to flatten the shape into a cross shape which is a common unwrapping:

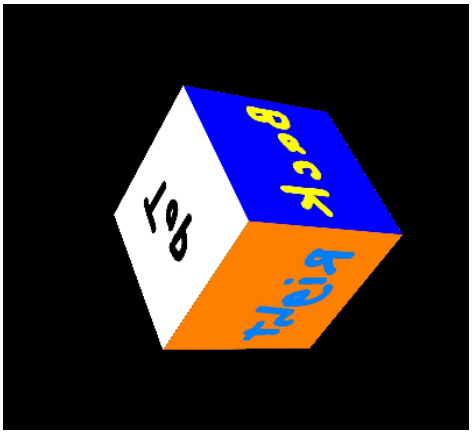


Figure 8: Cube with texture

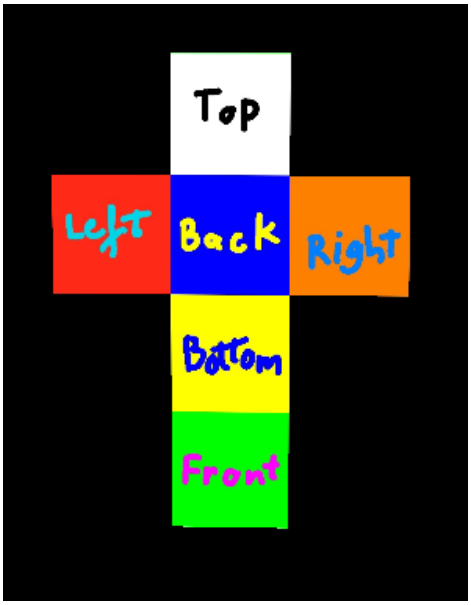


Figure 9: Cube unwrapped

III.4.5. Simple `ngonGenerator` and `quads2triangles` shapes (cylinder, cone, truncated cone, tetrahedron, prism)

All of these shapes are generated in a process similar to the Cube, as some combination of `ngonGenerator` for the top/bottom faces and a loop to generate quads as side walls:

- Cylinder: 2 n-gon top/bottom faces with a list of quads connecting the two
- Prism: 2 triangle top/bottom faces with a list of quads connecting the two
- Truncated cone: a Cylinder with 1 of the circles scaled down
- Cone: an n-gon pyramid, with an n-gon bottom face and a triangle fan as the side connecting to a vertex at the top
- tetrahedron: a specifically scaled cone with a triangle as a base

For `uvs`, we position a vertical slice, from pole to pole to form the left/right side of the uv map, as for the triangle fan on top and bottom, we place slices to split each triangle separately to beable to evenly split them along the top and bottom of the uv maps:

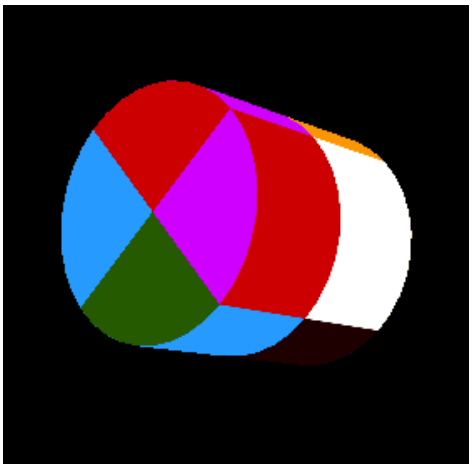


Figure 10: Cylinder with texture

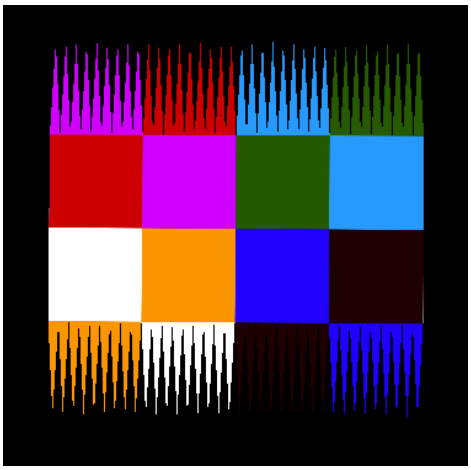


Figure 11: Cylinder unwrapped

III.4.6. Sphere

There are multiple ways to create a sphere, we chose uv sphere due to the similarity in construction with the earlier shapes.

In which, a sphere contains multiple `ngonGenerator` n-gons of various sizes and z offsets, akin to the latitudes of a globe. These rings are connected to adjacent loops akin to how the sides of a cylinder was generated, only for multiple levels. At the pole, the nearest n-gon to the pole vertex is connected similar to how the cone was generated.

Uv-Sphere's `uvs` is generated the same as the cylinder above. This allows us to use a mercator projection map as a texture to create a globe:

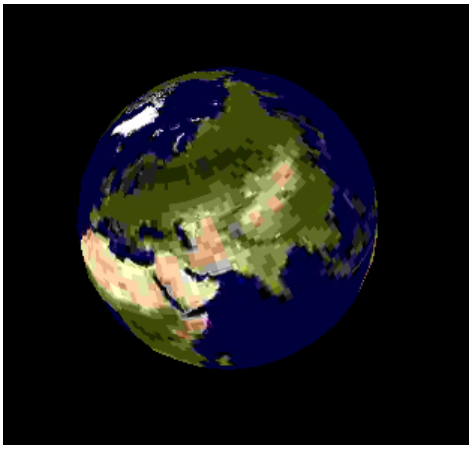


Figure 12: Globe

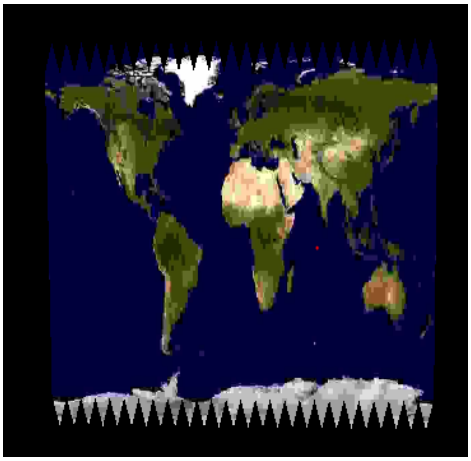


Figure 13: Globe unwrapped

III.4.7. Torus

To generate a Torus, we need to take into consideration its properties. For it roundedness, we can define it as cutting the torus in half (bisecting) through the plane containing the z axis (the center of the torus's hole) and have its cross section be a circle. The cross section of the torus should be a circle regardless of where you break it in half. In essence we are revolving a circle around some center axis (eg. the z axis in a 3d x y z coordinate system).

Because we are working with computers, it makes more sense to only generate a few slices, a few angles of the torus bisection to create a circular cross section, to avoid having to generate and render an infinite amount of vertices. So each "circular cross section" would be an `n-gon` shape and around the torus, there should only be `m` cross sections where `m` is a positive integer.

It is difficult to generate an `n-gon` cross-sections at different angles of bisection. So, instead of generating `m n-gon` cross sections, we can generate 1 piece of each `m` cross sections `n` times. To motivate this approach, imagine cutting the doughnut along the xy plane. You will see that the cross section now will be composed of 2 circles centered on the center of the torus' hole, one for the outside edge of the doughnut, one for the inner edge. We can think of making `m` pieces with an `ngonGenerator` generator producing `m-gon`s that are centered on the axis of revolution (the z axis). These `m-gon`s would have its radius and z offset borrowed from a point along the surface of the torus.

This means, in practice:

- we first generate a temporary `n-gon` offset from the z axis
- borrow the measurements of its vertices to generate our `m-gon`s making up the structure of our torus
- have some clever loops to connect these `m-gon`s' vertices together to form the surface of the torus

To generate the `uvs` we do a slice along an `n-gon` to define the left/right edges of the uv map, then do another slice along an `m-gon` to define the top/bottom of a uv map. The result is as follows:

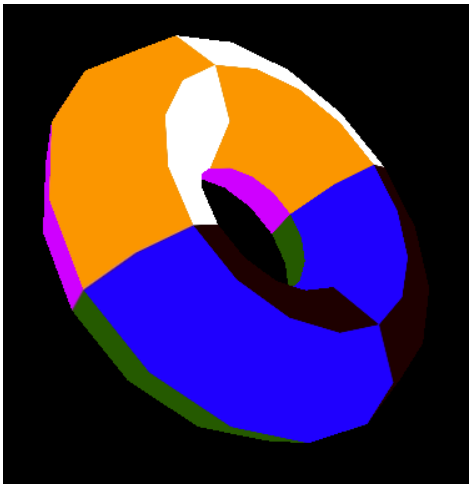


Figure 14: Torus with texture

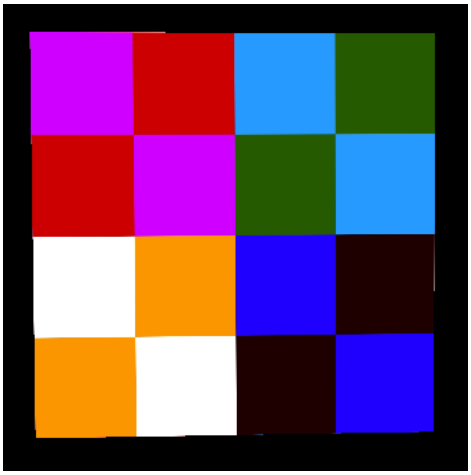


Figure 15: Torus unwrapped

Notice how the surface of the torus perfectly maps every point to a 2D surface where the top/bottom edge of the uv is connected and the left/right of the uv is connected.

III.4.8. Surface

Surface rendering takes in a function z = f(x,y) where f(x,y) is a python lambda: example:

```
func = lambda x, y: numpy.sin(x) + numpy.cos(y)
```

It also takes in parameters to control the limits of the surface to be shown in the x and y direction (`+-limx` and `+-limy`).

We consider a small slice of the xyz coordinate space where we want to render the surface of our function. It spans `+-limx` in the x direction, `+-limy` in the y direction and infinitely in the z direction. We populate it evenly with a vertex every dx and dy, and have that vertex's z offset be determined by the lambda function provided. Lastly, we can connect these vertices with their neighbors to create the surface of our function. Note that this implementation can not handle illegal (eg. divide by 0) or limits and can't render (or render accurately) discontinuous functions.

Since the `limx` and `limy` defines a rectangle area and the surface is continuous (by limitations), `uvs` are very simple to map almost 1 to 1

IV. Shaders

In a given scene, each object is associated with a shader program. A shader program is created from 2 shader: a vertex shader and a fragment shader. Specifically, when a program is created (in `Pipeline.update_program`), the 2 shader files are uploaded to the GPU before being compiled. They are then linked into a program, and the 2 former shader objects are destroyed.

However, the shaders might need some associated data: uniforms and textures. The `UManager class` is charged to handle these.

- uniforms:
  The uniforms are separated in 2 categories: static and dynamic.
  As the name suggests, the static ones stay the same during the whole execution of the program, while the dynamic ones change from frame to frame.
  There are currently only 2 dynamic uniforms: one to track the time passed from the start of the execution (`float iTime`), and another to track the mouse position (`vec4 iMouse`). Dynamic uniforms are computed for each frame and are declared in `windows.py`, thus this type of uniforms is not available for the final user. However, static uniforms can be defined by the user and passed as parameters in `Pipeline.add_object` during the creation of each object.
- textures:
  Currently textures are not supported.

V. Lighting

The lighting of a scene can completely change its final rendering, hence it plays a major role in computer graphics. In this project, we chose to handle scene lights thanks to a data structure that is passed to the fragment shader as a uniform. This uniform is in fact an array of lights, to enable the definition of multiple lights. A light is declared in python as an instance of the class `Light`, that is simply a container to hold the attributes defining the light.
Currently, the class is as follow:

```
class Light:
  def __init__(
    self,
    position,
    color=(1, 1, 1),
    intensity=1.0,
    # dtype=1,
    # constant=1.0,
    # linear=0.09,
    # quadratic=0.032,
  ):
    self.position = position
    self.color = color
    self.intensity = intensity
    # self.dtype = dtype
    # self.constant = constant
    # self.linear = linear
    # self.quadratic = quadratic
```

With the light passed down to the fragment shader, we simply have to apply a formula (Goureaud, Phong, …) to obtain the fragment color.

We also provide the support for materials by defining a set of default values for the different variables used during Phong or Goureaud rendering.

VI. GUI

The graphical user interface (GUI) is composed of a GL display window left and an option panel right.

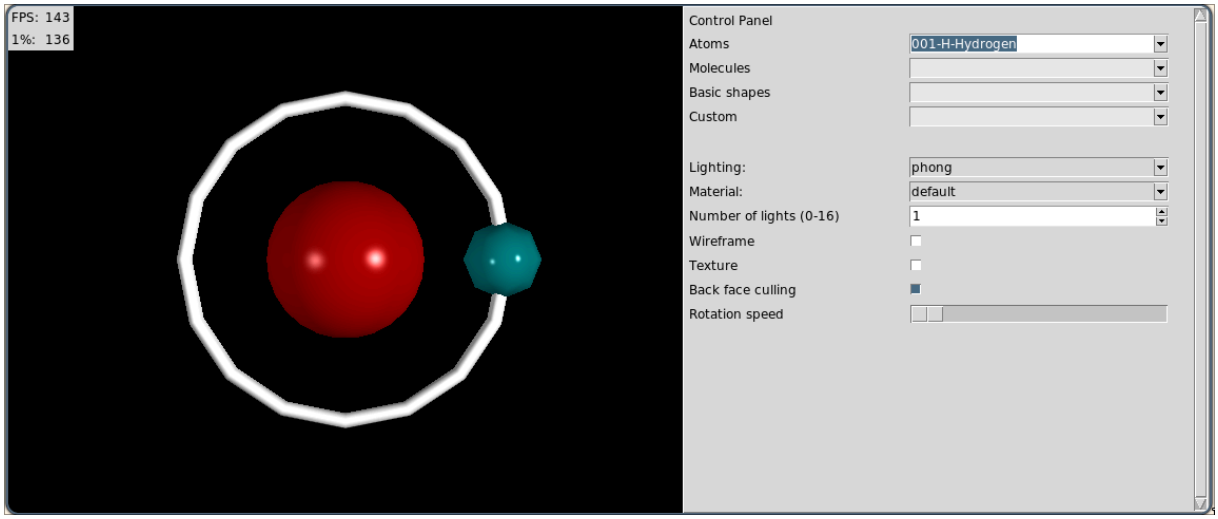


Figure 16: Application interface

The left panel displays the scene selected by the user on the left panel under the option “Displaying”. In the top left corner a little window indicates the frame displayed per second as well as the frame rate lowest 1% computed on a 1s time window. This panel supports mouse interaction through the use of camera: the user can change the direction

of the camera by dragging the mouse with left click, zoom with mouse wheel and change the camera position in the scene by dragging the mouse with right click.

The right panel provides the ability for the user to change the scene itself. They can select a predefined scene or import an `.obj` file under the "Scene" section. The "Lighting" option apply a given shading program to every object in the scene (some scenes may have objects with different programs). Other options allow the user to change the number of lights, enable wireframe mode or change the rotation speed of the scene.

VII. Challenges encountered

The integration of cameras inside the project architecture raised some questions about the project modularity and organisation. Indeed, cameras are needed to represent a 3D scene and one could decide to not provide any support from the library and ask the user to compute the view and projection matrices directly inside the vertex shader. This raises concerns about inputs, that then have to pass through uniforms, and about debugging. Indeed, debugging a Python code is much more convenient than debugging one written in GLSL, especially when it comes to matrix computations and hardware compatibility issues.

This reasons led us to create the `Camera` class, a pattern providing 3 main functions: `view_matrix`, `_projection_matrix` and `update`. The first 2 return the corresponding matrices and the latter returns an array of uniforms updated with the "projection" and "view" fields.

Then, once a camera implementation is passed to the main program, the user simply has to collect the "view" and "projection" uniforms in the vertex shader to compute the final vertices position.

List of Figures

Figure 1 Application displaying an hydrogen atom 3
Figure 2 Diagram of the software structure 4
Figure 3 Details of the scene selection 4
Figure 4 General shape generation flow 7
Figure 5 Normals generator 11
Figure 6 Slicer diagram (simple) 11
Figure 7 Slicer diagram (detailed) 12
Figure 8 Cube with texture 13
Figure 9 Cube unwrapped 13
Figure 10 Cylinder with texture 14
Figure 11 Cylinder unwrapped 14
Figure 12 Globe 14
Figure 13 Globe unwrapped 14
Figure 14 Torus with texture 15
Figure 15 Torus unwrapped 15
Figure 16 Application interface 17